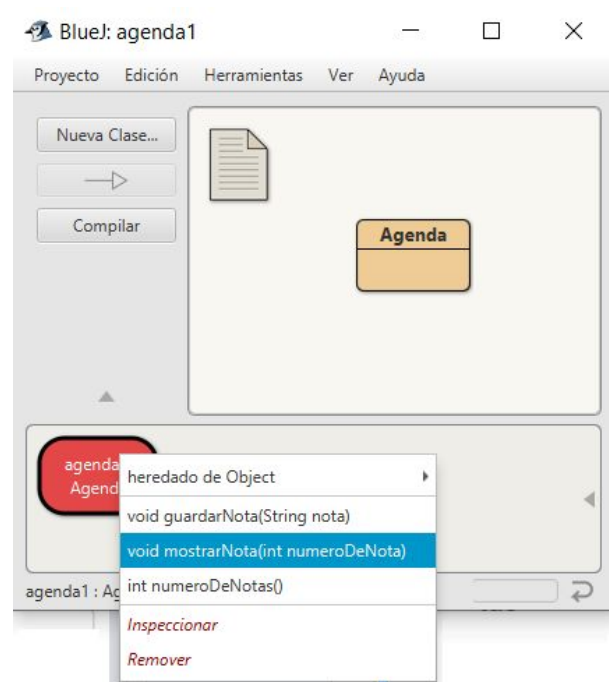
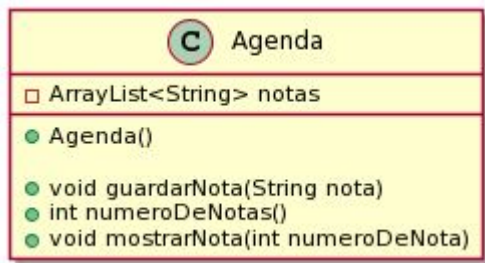


Informática Avanzada

Agrupamiento de Objetos -
Colecciones

Proyecto *agenda*

- Exploramos el proyecto *agenda1*
- El sistema representa una agenda
- Contiene notas (que son textos, Strings)
- Una vez creado una Agenda, se puede
 - agregar notas
 - mostrar una nota
 - determinar cuántas notas contiene



Bibliotecas de clases

- Las librerías contienen clases predefinidas
- Estas clases están agrupadas en paquetes (packages) para resolver problemas comunes
 - `java.util`
 - `java.math`
 - `java.io`
- Se puede importar todo un paquete o clases individuales en nuestro sistema
 - `import java.util.*;`
 - `import java.io.File;`
- El package ***java.lang*** contiene clases de uso habitual (e.g., `String`) y es importado por defecto



docs.oracle.com/javase/10/docs/api/index.html?overview-summary.html

← → ↺

🔍

☆

☰

Incógnita

java.time.chrono
java.time.format
java.time.temporal
java.time.zone
java.util
java.util.concurrent
java.util.concurrent.atomic
java.util.concurrent.locks
java.util.function
java.util.jar
java.util.logging
java.util.prefs
java.util.regex
java.util.spi
java.util.stream
java.util.zip
javafx.animation

Set
SortedMap
SortedSet
Splitter
Splitter.OfDouble
Splitter.OfInt
Splitter.OfLong
Splitter.OfPrimitive

Classes
AbstractCollection
AbstractList
AbstractMap
AbstractMap.SimpleEntry
AbstractMap.SimpleImmutableEntry
AbstractQueue
AbstractSequentialList
AbstractSet
ArrayDeque
ArrayList
Arrays
Base64
Base64.Decoder
Base64.Encoder
BitSet
Calendar
Calendar.Builder
Collections
Currency
Date
Dictionary
DoubleSummaryStatistics
EnumMap
EnumSet
EventListenerProxy
EventObject
FormatableFlags
Formatter
GregorianCalendar
HashMap

OVERVIEW MODULE PACKAGE **CLASS** USE TREE DEPRECATED INDEX HELP

PREV CLASS NEXT CLASS **FRAMES** NO FRAMES

SUMMARY: NESTED | FIELD | CONSTRUCTOR | METHOD DETAIL: FIELD | CONSTR | METHOD

SEARCH:

Constructor Summary

Constructors

Constructor	Description
<code>ArrayList()</code>	Constructs an empty list with an initial capacity of ten.
<code>ArrayList(int initialCapacity)</code>	Constructs an empty list with the specified initial capacity.
<code>ArrayList(Collection<? extends E> c)</code>	Constructs a list containing the elements of the specified collection, in the order they are returned by the collection's iterator.

Method Summary

All Methods Instance Methods Concrete Methods

Modifier and Type	Method	Description
void	<code>add(int index, E element)</code>	Inserts the specified element at the specified position in this list.
boolean	<code>add(E e)</code>	Appends the specified element to the end of this list.
boolean	<code>addAll(int index, Collection<? extends E> c)</code>	Inserts all of the elements in the specified collection into this list, starting at the specified position.
boolean	<code>addAll(Collection<? extends E> c)</code>	Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's iterator.
void	<code>clear()</code>	Removes all of the elements from this list.
Object	<code>clone()</code>	Returns a shallow copy of this ArrayList instance.
boolean	<code>contains(Object o)</code>	Returns true if this list contains the specified element.
void	<code>ensureCapacity(int minCapacity)</code>	Increases the capacity of this ArrayList instance, if necessary, to ensure that it can hold at least the number of elements specified by the minimum capacity argument.
void	<code>forEach(Consumer<? super E> action)</code>	Performs the given action for each element of the Iterable until all elements have been processed or the action throws an exception.
E	<code>get(int index)</code>	Returns the element at the specified position in this list.
int	<code>indexOf(Object o)</code>	Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element.

docs.oracle.com/en/java/javase/11/docs/api/index.html

OVERVIEWMODULEPACKAGECLASSUSE TREEDEPRECATEDINDEXHELP

ALL CLASSES

Java® Platform, Standard Edition & Java Development Kit
Version 11 API Specification

This document is divided into two sections:

Java SE

The Java Platform, Standard Edition (Java SE) APIs define the core Java platform for general-purpose computing. These APIs are in modules whose names start with java.

JDK

The Java Development Kit (JDK) APIs are specific to the JDK and will not necessarily be available in all implementations of the Java SE Platform. These APIs are in modules

All ModulesJava SEJDKOther Modules

Module	Description
java.base	Defines the foundational APIs of the Java SE Platform.
java.compiler	Defines the Language Model, Annotation Processing, and Java Compiler APIs.
java.datatransfer	Defines the API for transferring data between and within applications.
java.desktop	Defines the AWT and Swing user interface toolkits, plus APIs for accessibility, audio, imaging, printing, and JavaBeans.
java.instrument	Defines services that allow agents to instrument programs running on the JVM.
java.logging	Defines the Java Logging API.
java.management	Defines the Java Management Extensions (JMX) API.
java.management.rmi	Defines the RMI connector for the Java Management Extensions (JMX) Remote API.
java.naming	Defines the Java Naming and Directory Interface (JNDI) API.
java.net.http	Defines the HTTP Client and WebSocket APIs.
java.prefs	Defines the Preferences API.
java.rmi	Defines the Remote Method Invocation (RMI) API.
java.scripting	Defines the Scripting API.
java.se	Defines the API of the Java SE Platform.
java.security.jgss	Defines the Java binding of the IETF Generic Security Services API (GSS-API).
java.security.sasl	Defines Java support for the IETF Simple Authentication and Security Layer (SASL).

SEARCH: java.util

Packages

java.base/java.util
java.base/java.util.concurrent
java.base/java.util.concurrent.atomic
java.base/java.util.concurrent.locks
java.base/java.util.function
java.base/java.util.jar
java.logging/java.util.logging
java.prefs/java.util.prefs
java.base/java.util.regex
java.base/java.util.spi
java.base/java.util.stream
java.base/java.util.zip

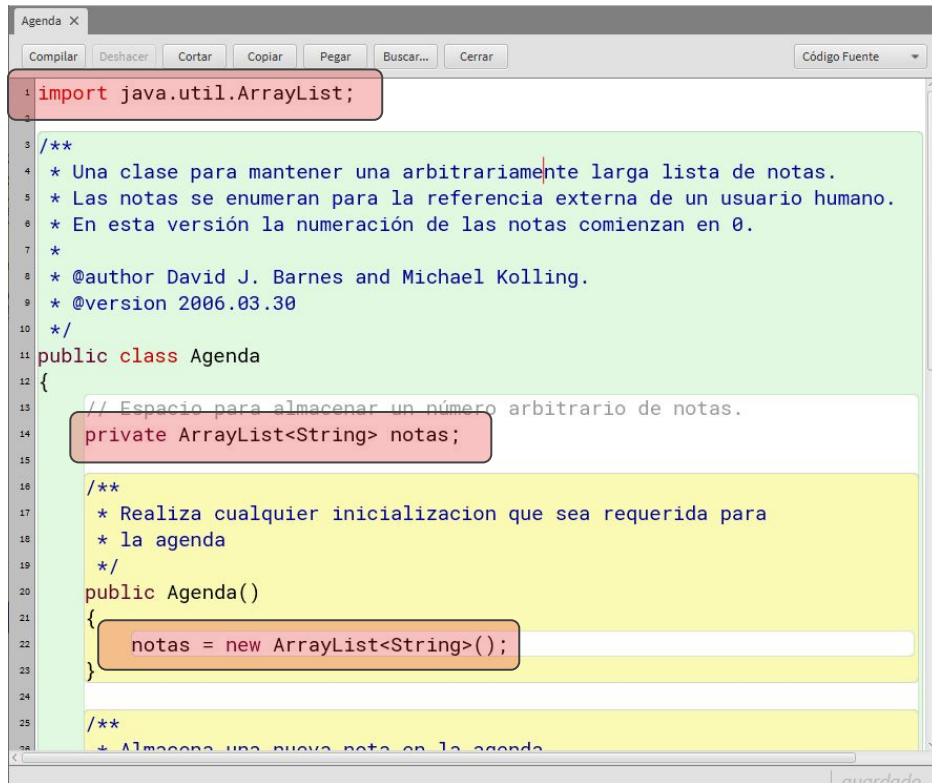
Types

java.util.concurrent.ThreadPoolExecutor.AbortPolicy
java.util.AbstractCollection
java.util.Spliterators.AbstractDoubleSpliterator
java.util.concurrent.AbstractExecutorService
java.util.Spliterators.AbstractIntSpliterator
java.util.AbstractList
java.util.Spliterators.AbstractLongSpliterator
java.util.AbstractMap
java.util.concurrent.locks.AbstractOwnableSynchronizer
java.util.prefs.AbstractPreferences
java.util.AbstractQueue
java.util.concurrent.locks.AbstractQueuedLongSynchronizer
java.util.concurrent.locks.AbstractQueuedSynchronizer
java.util.spi.AbstractResourceBundleProvider
java.util.AbstractSequentialList
java.util.AbstractSet
java.util.Spliterators.AbstractSpliterator
java.util.zip.Adler32
java.util.concurrent.ArrayBlockingQueue
java.util.ArrayDeque
java.util.ArrayList

Preferencias sobre cookies | Ad Choices

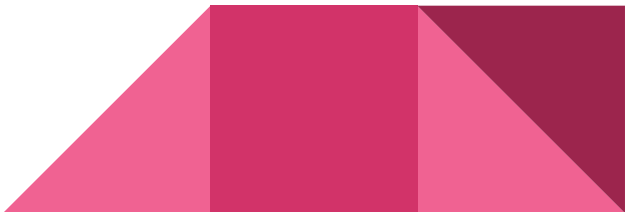
Proyecto “Agenda Personal”

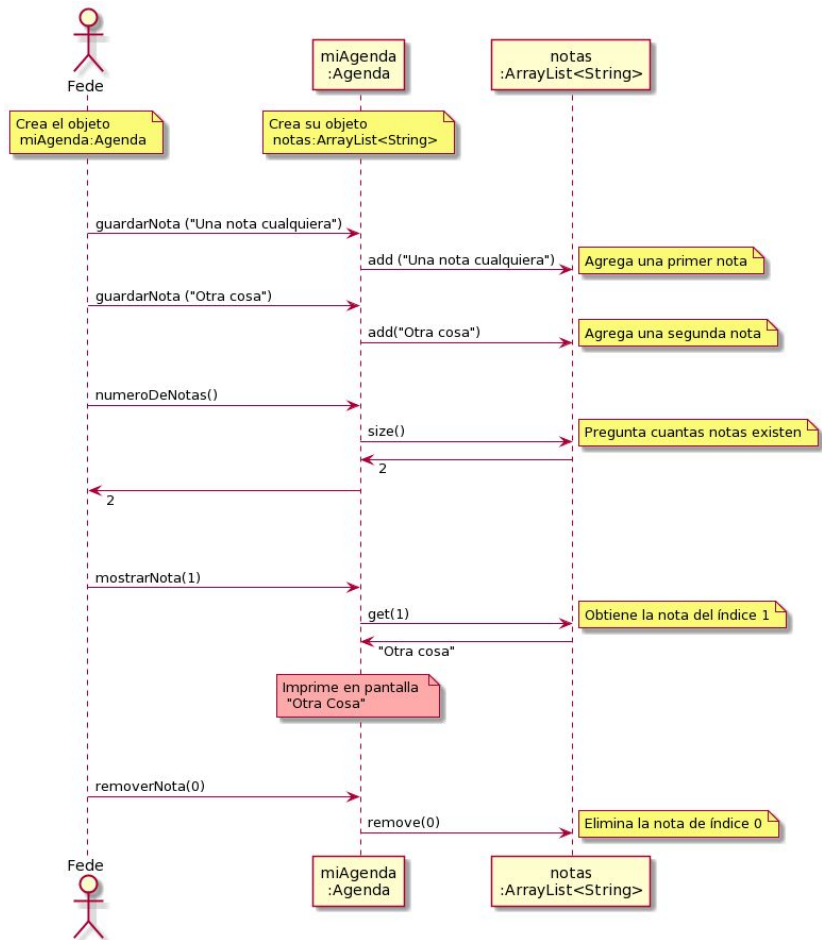
- Importar la clase **ArrayList** del paquete **java.util**
 - **import** java.util.ArrayList;
- (o importar el paquete completo → **import** java.util.*;)
- Definir una variable para un **ArrayList** de **String**
 - **ArrayList<String>** notas;
- Crear un objeto **ArrayList** de **String** y asignarlo a una variable
 - **notas = new ArrayList<String>();**

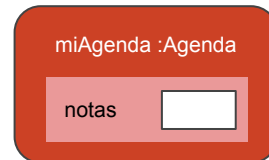
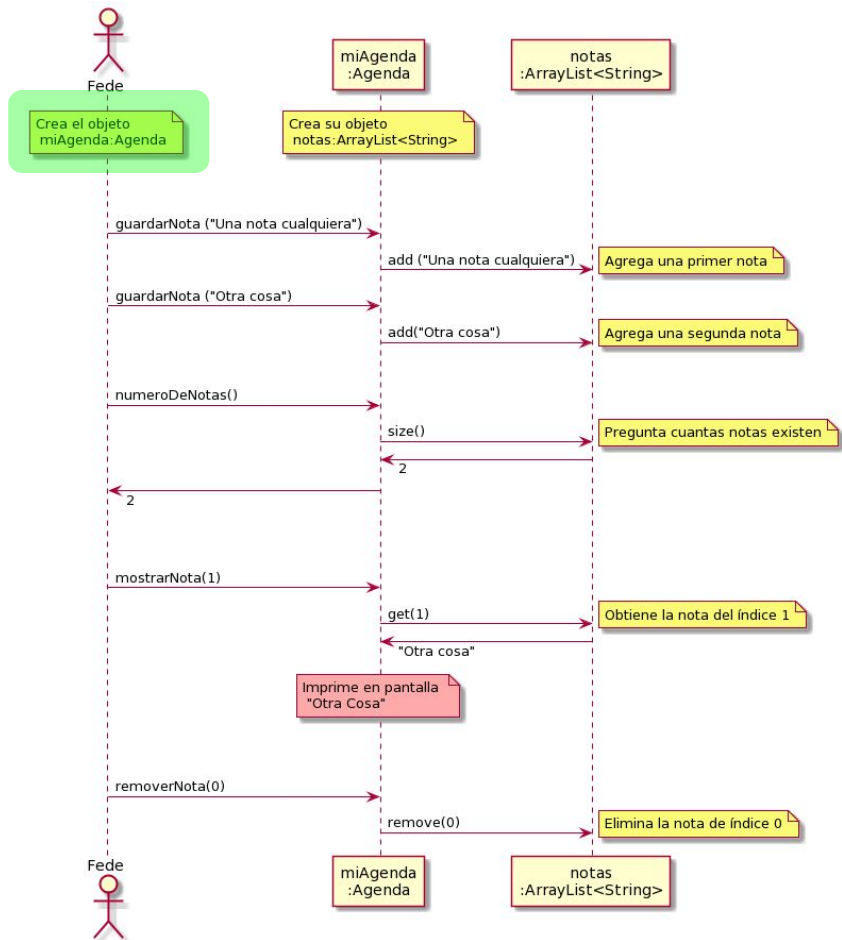


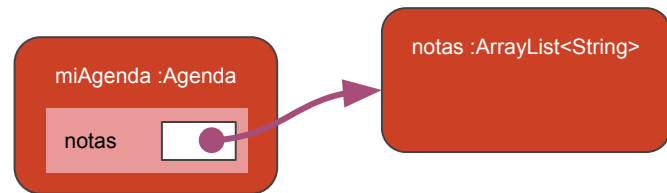
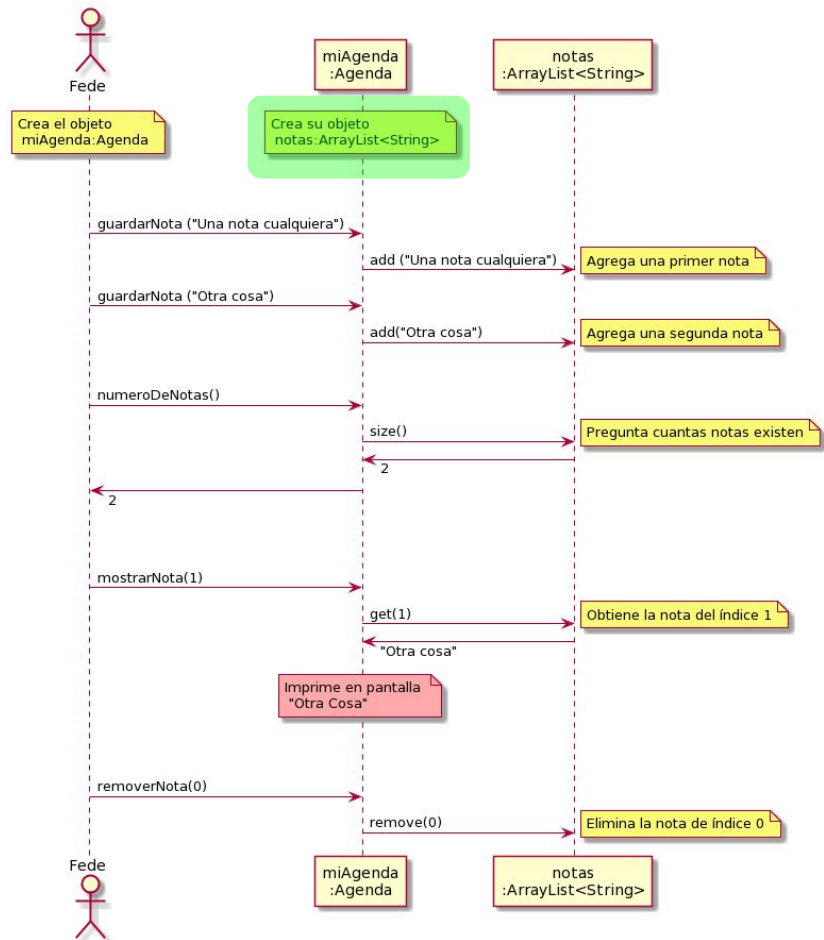
```
1 import java.util.ArrayList;
2
3 /**
4  * Una clase para mantener una arbitrariamente larga lista de notas.
5  * Las notas se enumeran para la referencia externa de un usuario humano.
6  * En esta versión la numeración de las notas comienzan en 0.
7  *
8  * @author David J. Barnes and Michael Kolling.
9  * @version 2006.03.30
10 */
11 public class Agenda
12 {
13     // Espacio para almacenar un número arbitrario de notas.
14     private ArrayList<String> notas;
15
16     /**
17      * Realiza cualquier inicializacion que sea requerida para
18      * la agenda
19      */
20     public Agenda()
21     {
22         notas = new ArrayList<String>();
23     }
24
25     /**
26      * Almacena una nueva nota en la agenda
```

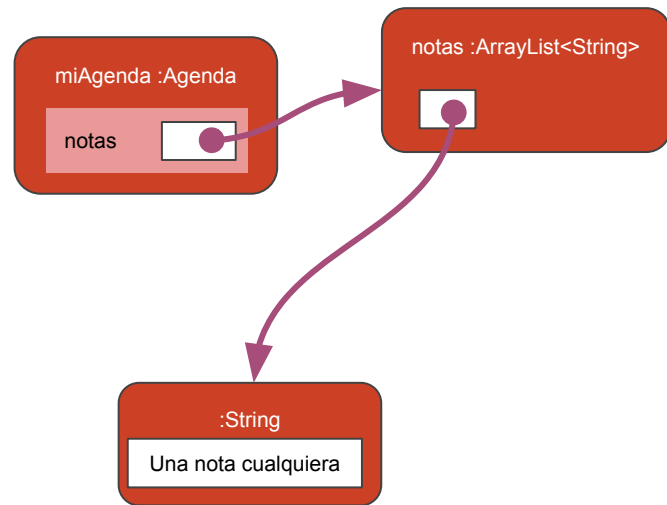
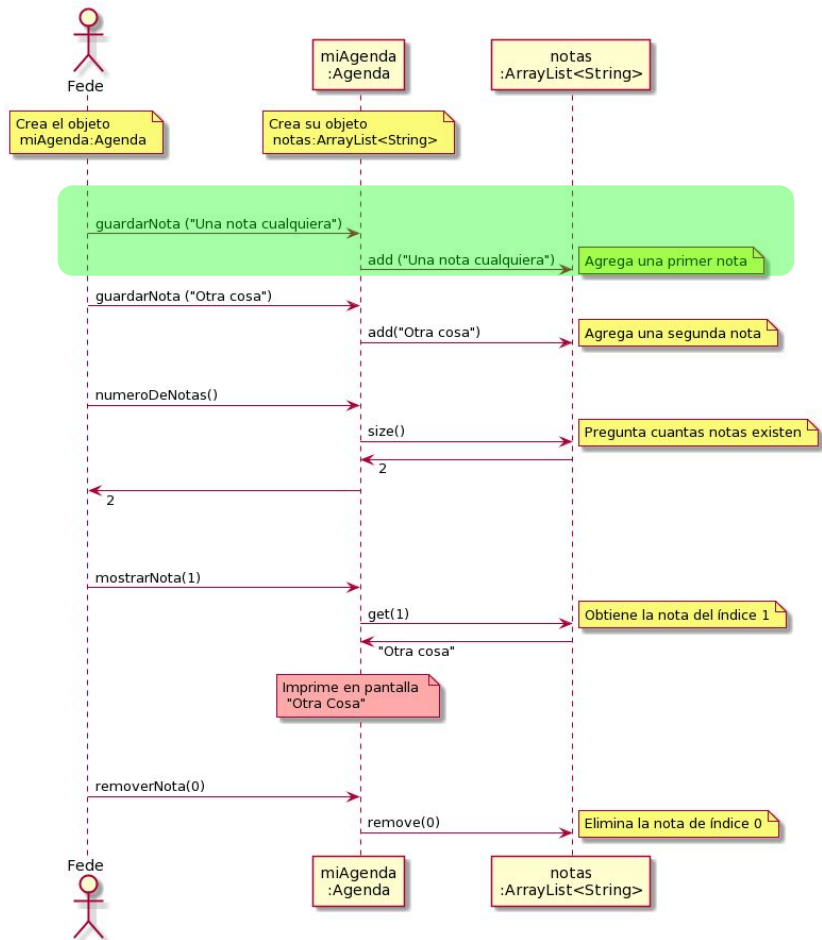
ArrayList

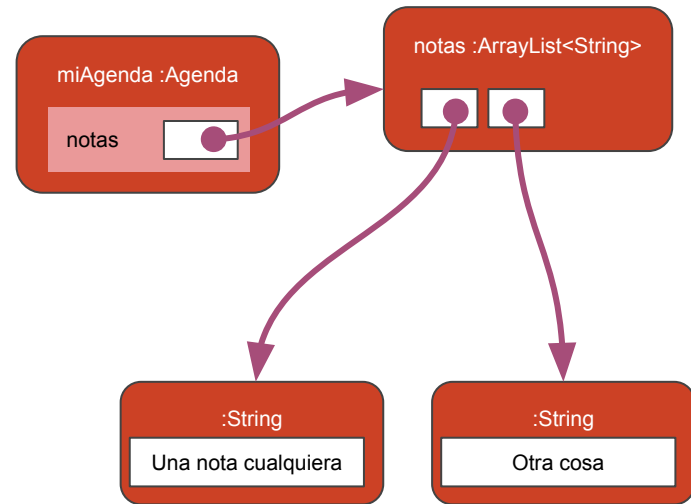
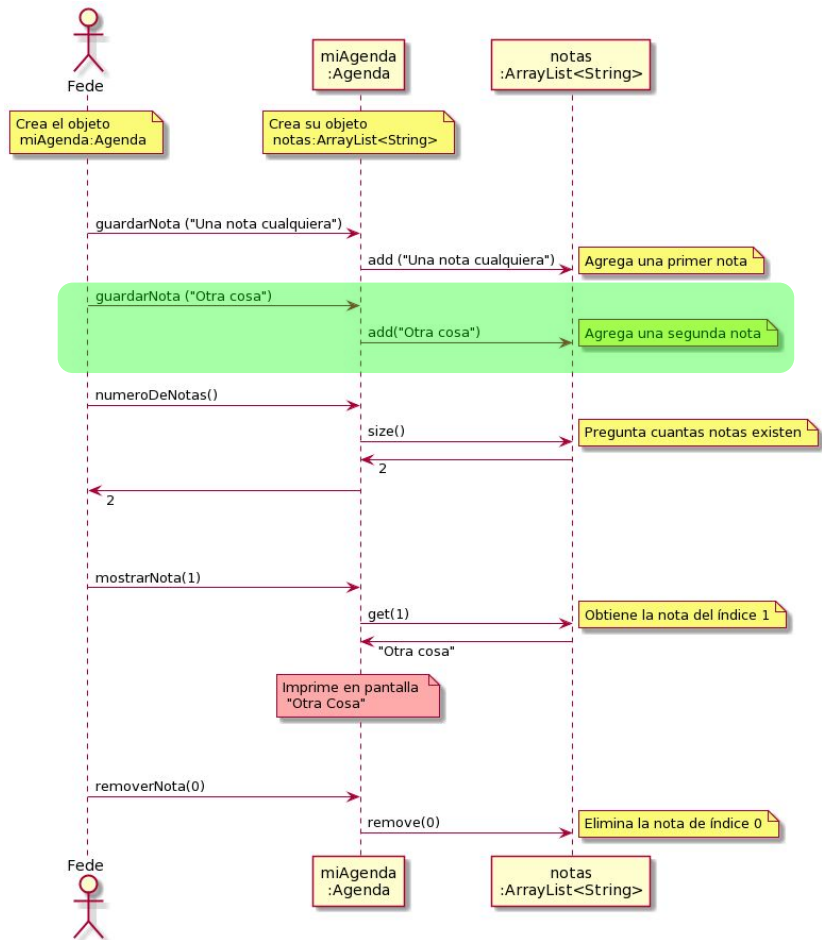
- Es una **colección**
 - Mantiene objetos agrupados en una **lista**
 - Es “parametrizada”: requiere que **se especifique qué va a contener** la lista
 - ArrayList <String>
 - ArrayList <Estudiante>
 - ArrayList <Circulo>
 - Pueden **agregarse** o **remove**se elementos de la lista
 - El **tamaño** de la lista es **dinámico**
 - Cada elemento tiene un **índice**, que puede variar
 - Los **métodos básicos** para interactuar con la lista son
 - add, remove, get, size
 - Más referencias en la API de JAVA.
- 

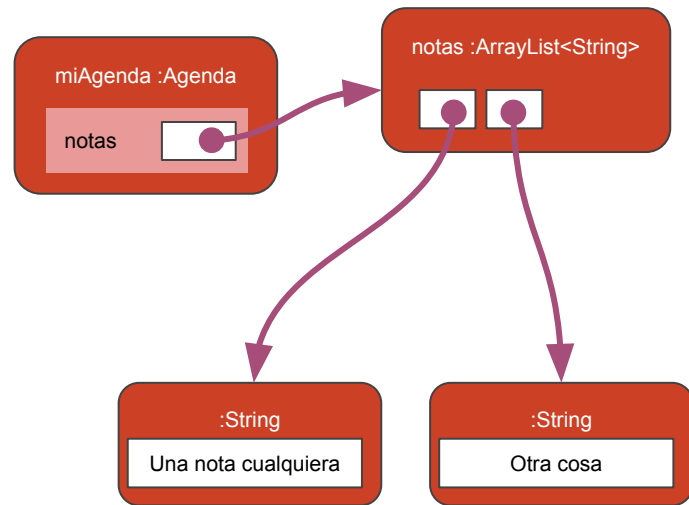
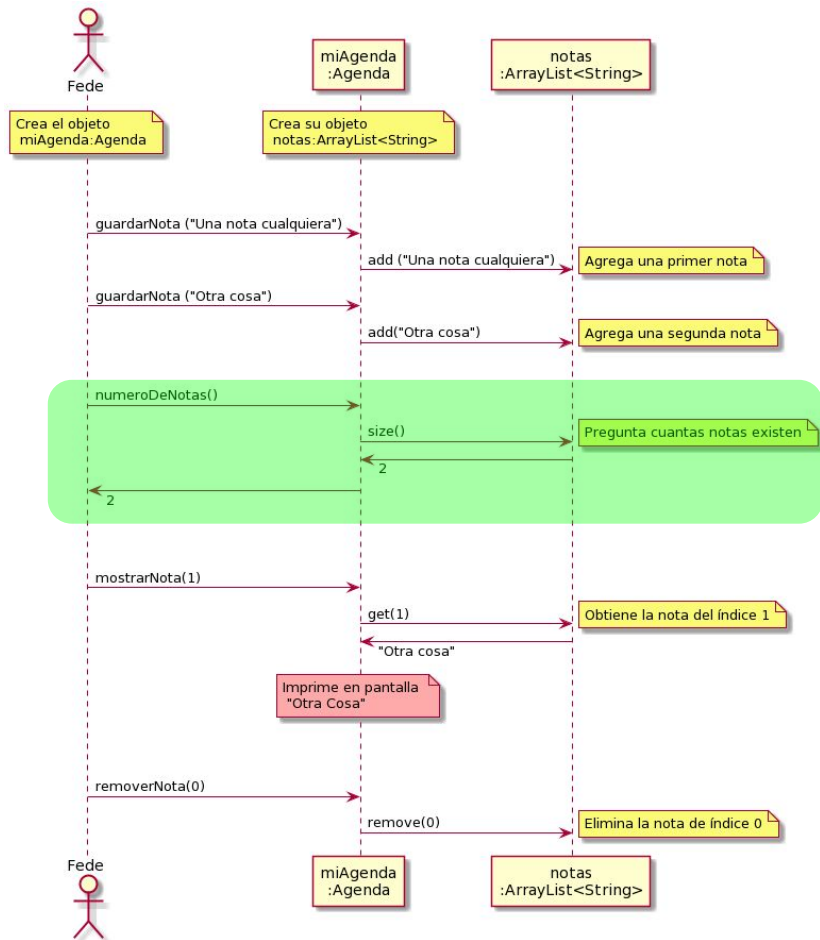


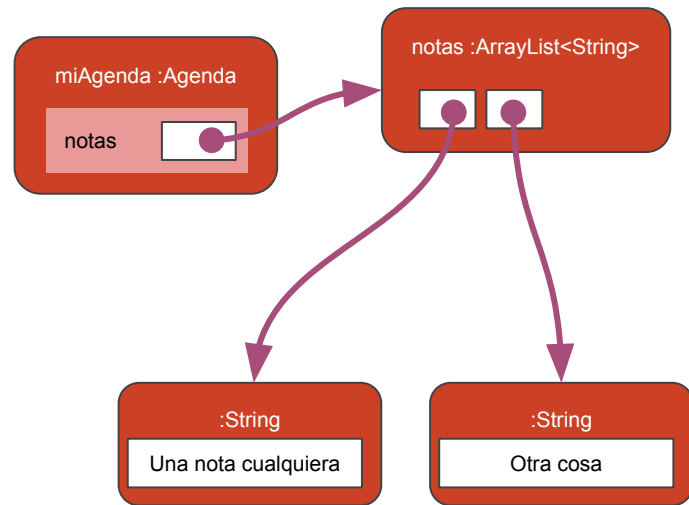
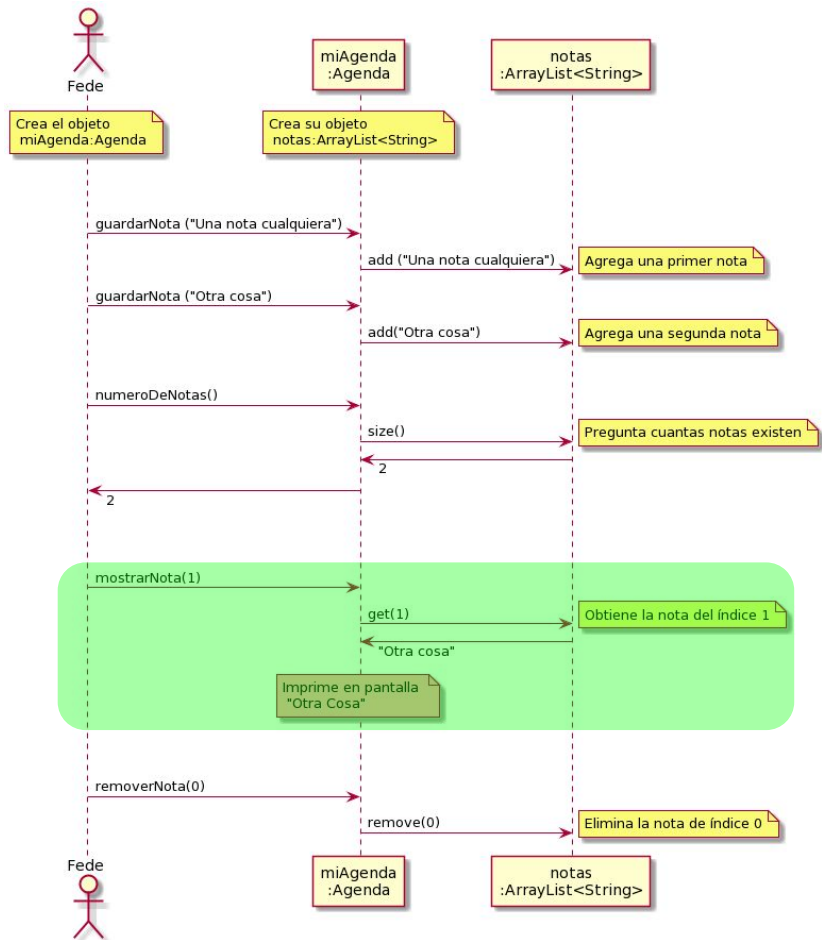


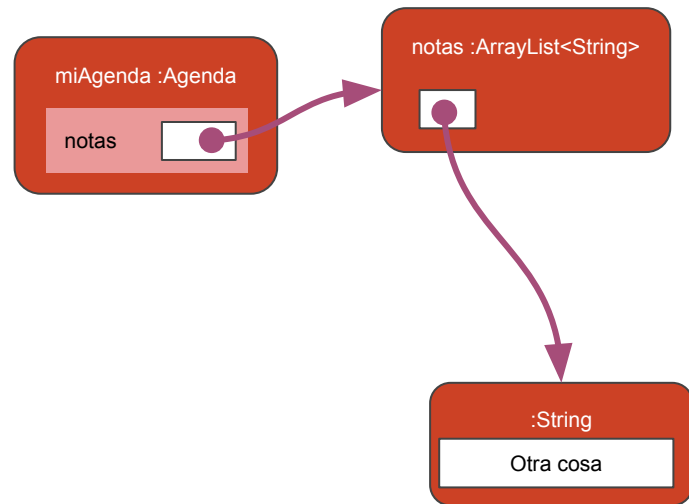
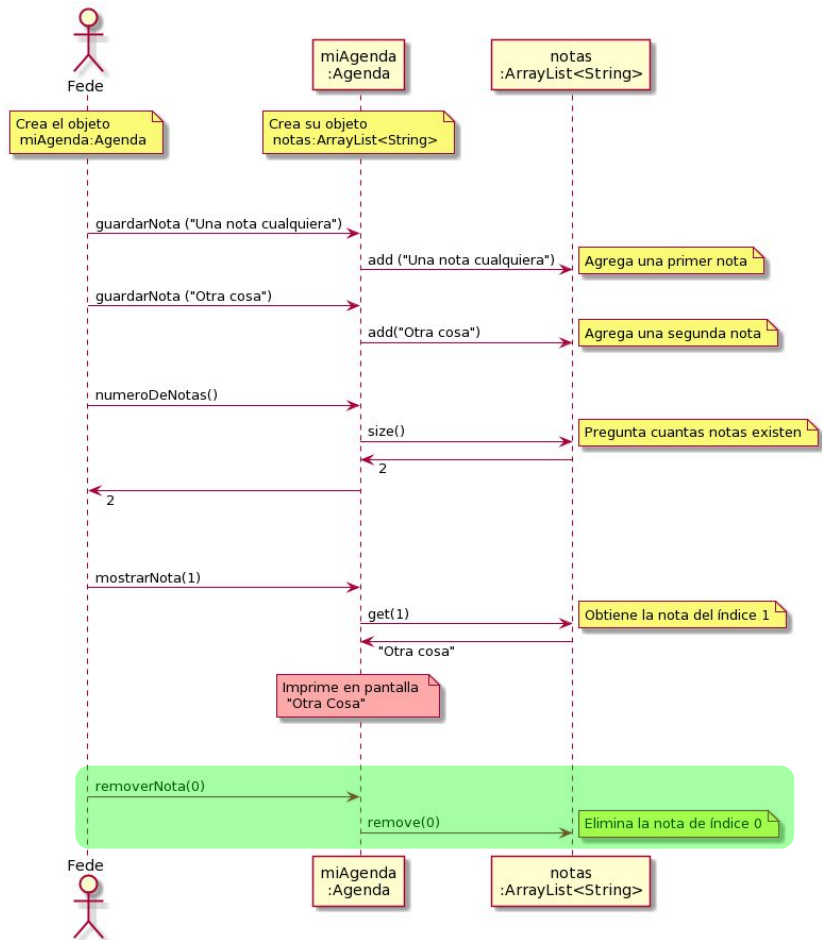












Informática Avanzada

Iteración

Iteración

- Es el concepto de repetir una acción o ejecutar un bloque de código varias veces
- Los bucles permiten controlar cuántas veces repetir una acción
- En las colecciones, queremos ejecutar la misma acción para cada objeto
- Existen instrucciones de repetición
 - for
 - for-each
 - while



for each - Sintaxis

```
for (TipoDeElemento elem: coleccion)  
{  
    // Cuerpo del ciclo  
}
```

```
/**  
 * Lista todas las notas en la agenda.  
 */  
public void imprimirNotas()  
{  
    for(String text : notas) {  
        System.out.println(text);  
    }  
}
```

- Declara una variable local **elem** del tipo *TipoDeElemento*,
- En **elem** se asignan de a uno por vez, cada objeto que existe en la **coleccion**,
- Se ejecuta el cuerpo del ciclo para cada uno de los objetos de la coleccion

for & while - Sintaxis

```
for(<inicializacion>; <expresión lógica>; <acción post-cuerpo>)  
{  
    // Cuerpo del ciclo  
}
```

```
while (<expresión lógica>)  
{  
    // Cuerpo del ciclo  
}
```

```
for (int i=0; i<10; i++)  
{  
    System.out.println(i);  
}
```

```
int i=0;  
while (i<10)  
{  
    System.out.println(i);  
    i++;  
}
```

- Permiten repetir la ejecución de un bloque de código
- No están limitados al uso con colecciones
- Pueden recorrer una colección accediendo a sus elementos a través de un índice
- El ciclo **for** ejecuta un número fijo de iteraciones
- El ciclo **while** ejecuta un número indefinido de iteraciones
 - Hay que prestar atención a bucles infinitos

Iterator

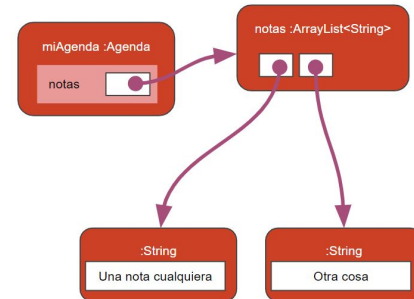
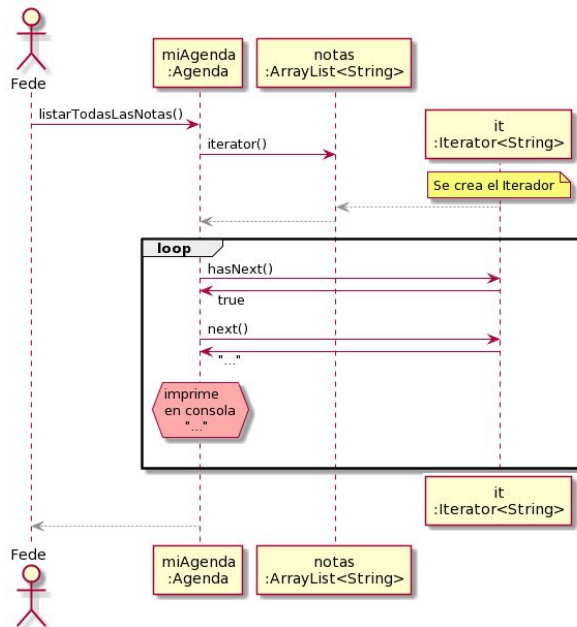
- Un iterador es un objeto que permite recorrer los elementos de una colección
- Es creado por una colección.
- Sólo se puede pedirle dos cosas
 - que diga si en la colección que lo creó, ya se recorrieron todos los elementos (***hasNext()***)
 - que entregue una objeto (**una referencia a un objeto**) que aún no haya sido recorrido (***next()***)
- A través de un ciclo ***while*** puede recorrer todos los elementos de la colección



Usar un objeto iterador

- Importar la clase *Iterator*
 - **import** java.util.Iterator;
- Declarar variable para referenciar al iterador
 - **Iterator** <TipoDeDato> it;
- Obtener el objeto iterador de la colección
 - **it = <coleccion>.iterator()**
- Usar el iterador para determinar si hay elementos
 - **while (it.hasNext())**
- Obtener los elementos de la colección
 - { **it.next();** // hacer algo
 - }

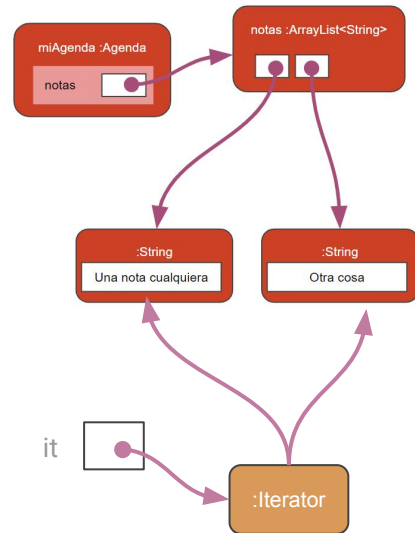
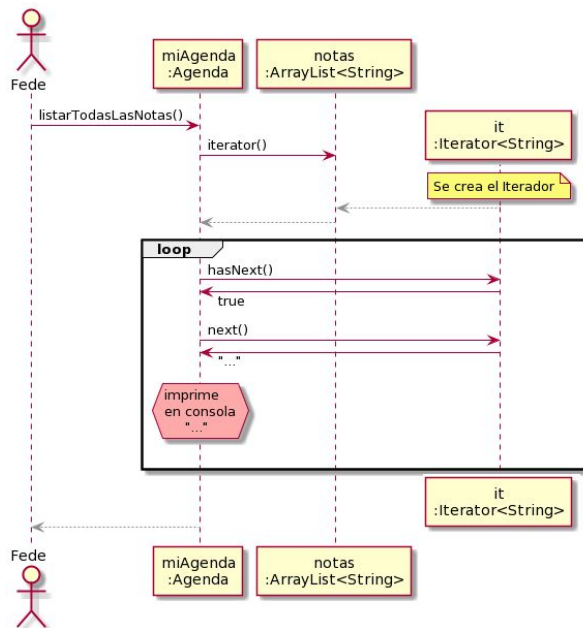
```
public void listarTodasLasNotas()
{
    Iterator<String> it = notas.iterator();
    while(it.hasNext()) {
        System.out.println(it.next());
    }
}
```



Usar un objeto iterador

- Importar la clase *Iterator*
 - **import** java.util.Iterator;
- Declarar variable para referenciar al iterador
 - **Iterator** <TipoDeDato> it;
- Obtener el objeto iterador de la colección
 - **it = <coleccion>.iterator()**
- Usar el iterador para determinar si hay elementos
 - **while (it.hasNext())**
- Obtener los elementos de la colección
 - { **it.next();** // hacer algo
 - }

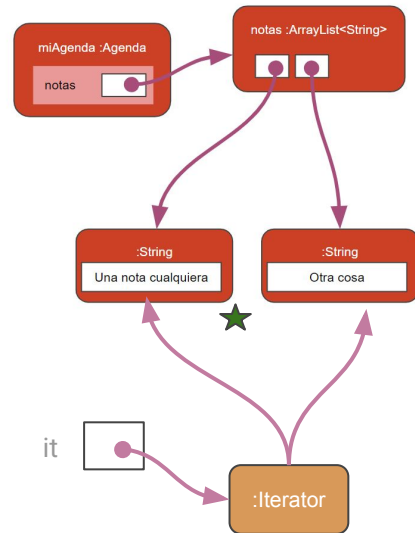
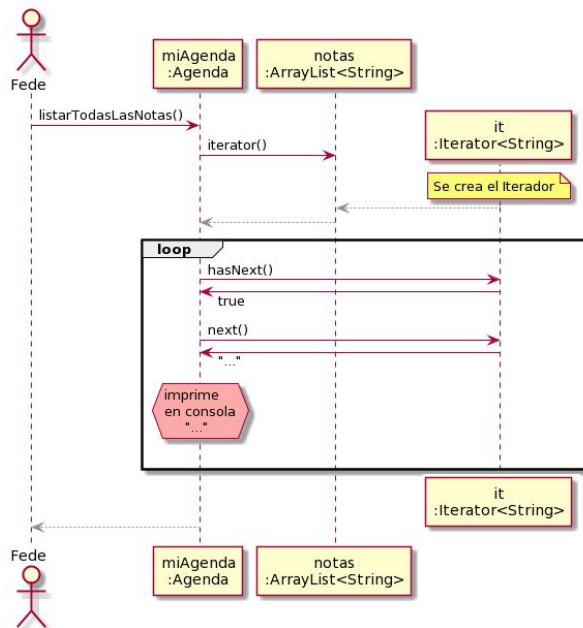
```
public void listarTodasLasNotas()
{
    Iterator<String> it = notas.iterator();
    while(it.hasNext()) {
        System.out.println(it.next());
    }
}
```



Usar un objeto iterador

- Importar la clase *Iterator*
 - **import java.util.Iterator;**
- Declarar variable para referenciar al iterador
 - **Iterator <TipoDeDato> it;**
- Obtener el objeto iterador de la colección
 - **it = <coleccion>.iterator()**
- Usar el iterador para determinar si hay elementos
 - **while (it.hasNext())**
- Obtener los elementos de la colección
 - **it.next();** // hacer algo

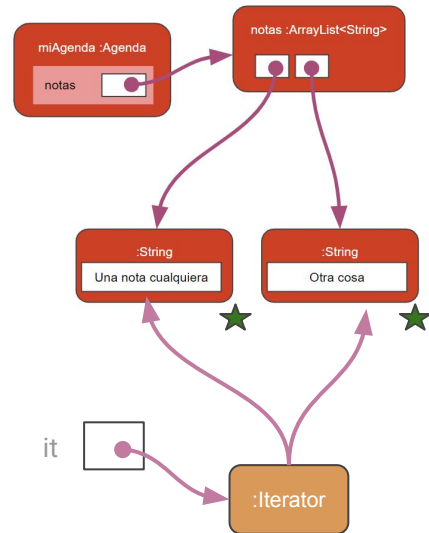
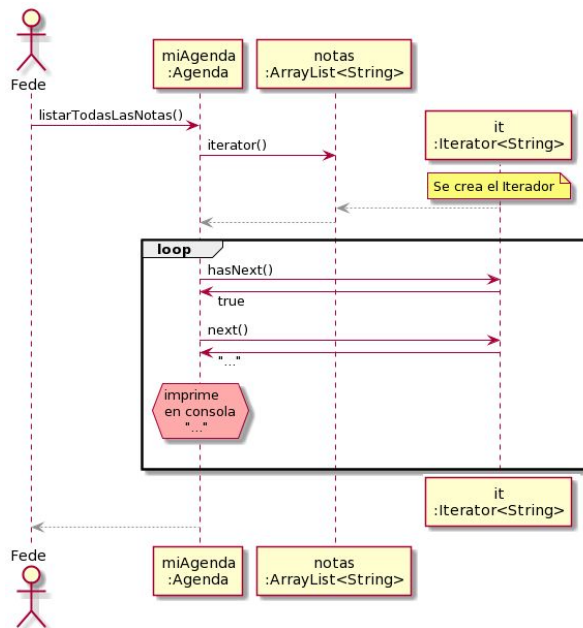
```
public void listarTodasLasNotas()
{
    Iterator<String> it = notas.iterator();
    while(it.hasNext()) {
        System.out.println(it.next());
    }
}
```



Usar un objeto iterador

- Importar la clase *Iterator*
 - **import** java.util.Iterator;
- Declarar variable para referenciar al iterador
 - **Iterator** <TipoDeDato> it;
- Obtener el objeto iterador de la colección
 - **it = <coleccion>.iterator()**
- Usar el iterador para determinar si hay elementos
 - **while (it.hasNext())**
- Obtener los elementos de la colección
 - **it.next();** // hacer algo

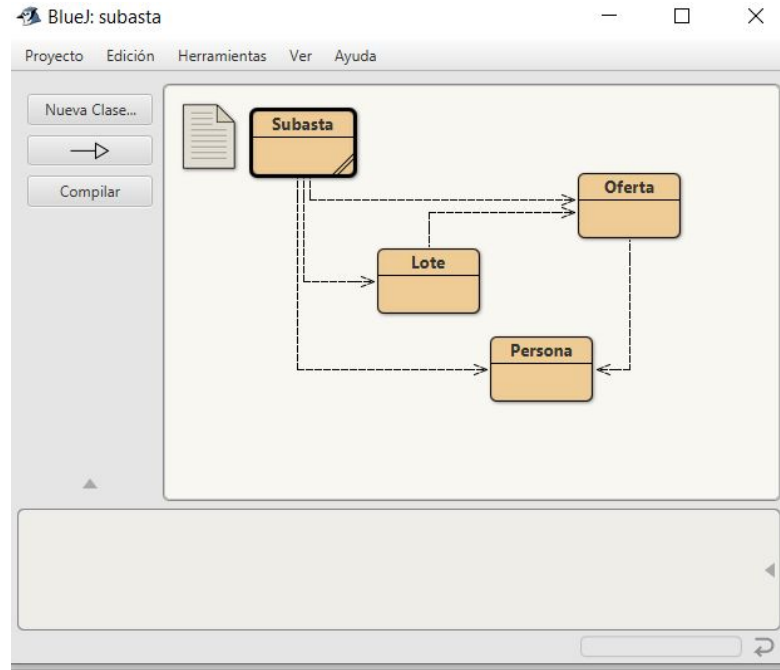
```
public void listarTodasLasNotas()
{
    Iterator<String> it = notas.iterator();
    while(it.hasNext()) {
        System.out.println(it.next());
    }
}
```



Índices vs. Iteradores

- Ciclo ***for-each***
 - Solo aplica para iterar elementos de una colección
 - Procesa **todos** los elementos de la colección
 - Permite recorrer los elementos sin el uso de índice
- Ciclo ***while***
 - Itera un número **indeterminado** de veces,
 - La iteración es controlada por una expresión lógica
 - Puede usarse para recorrer una colección con acceso indexado.
 - Su uso no está limitado a recorrer colecciones
- Ciclo ***for***
 - Itera un número **fijo** de veces
 - Puede usarse para recorrer una colección con acceso indexado.
 - Su uso no está limitado a recorrer colecciones
- Usando ***iterator***
 - Permite acceder secuencialmente a todos los elementos de la colección.
 - No requiere el uso de índice para acceder a los elemento
 - Se usa en combinación con ***while***

El proyecto *subasta*





Informática Avanzada

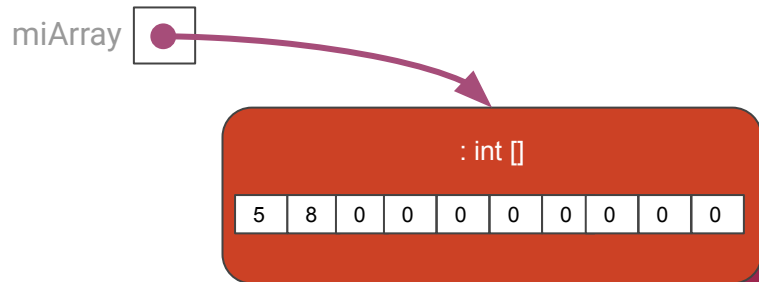
Otras Colecciones

Colecciones de tamaño Fijo.

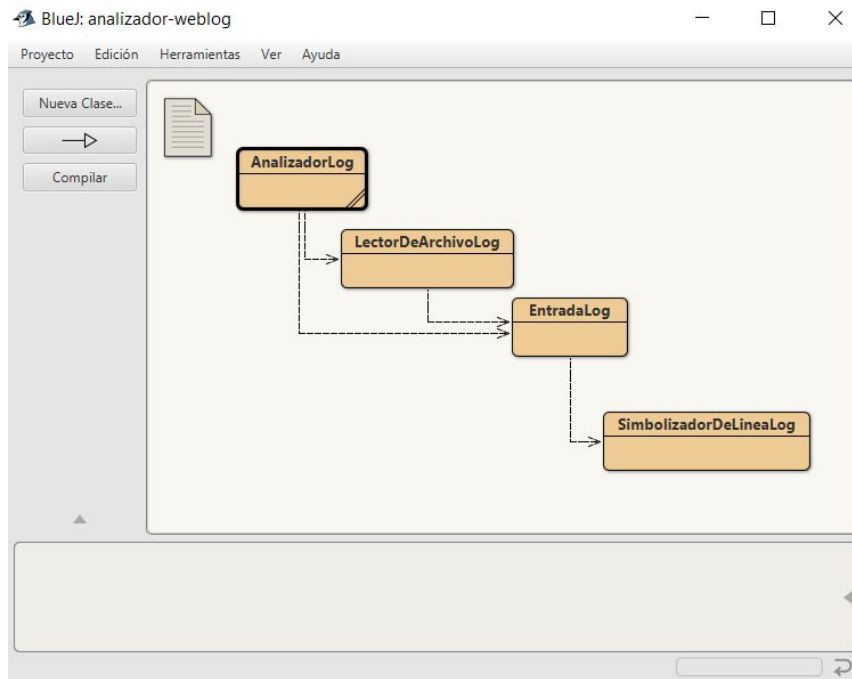
- Arrays (o arreglos) son estructuras de datos con elementos del mismo tipo
- Sus elementos pueden ser tanto objetos como valores primitivos
- Tienen un tamaño fijo, definido al momento de declararlos
- Sus elementos siempre están indexados
- Se utiliza la notación de corchetes para acceder éstos.

```
int[] miArray;           //Declaración del array
miArray=new int[10];     //Creación del array
miArray[0] = 5;          //Operación de asignación
miArray[1] = miArray[0] + 3; //Acceso a un elemento y asignación

for (int index=0; index<miArray.length; index++) //Obtener el tamaño
{
    //Recorremos el array e imprimimos sus elementos
    System.out.println (miArray[index]);
}
```



El proyecto *analizador-weblog*



Otras Colecciones

- ArrayList
- LinkedList
- HashSet
- HashMap
- TreeMap



: Lote

int numero	1
String desc	Peugeto
Oferta ofertaMax	null

visorDeR1: VisorDeReloj

private VisorDeNumeros horas	
private VisorDeNumeros minutos	
private String cadVisor	

:ItemDeCorreo

de	Ana
para	Juan
mensaje	Hola

